

25: Reinforcement Learning for Language Models

Date: April 23, 2026

Acknowledgements. These notes draw on the framing in Damek Davis’s STAT 4830 notes (in particular the decision to bypass most of the classical reinforcement learning formalism) and on Nathan Lambert’s *RLHF Book* [Lam25] for the RLHF chapter.

1 From pretraining to post-training

Last lecture we trained a language model by maximum likelihood: given a corpus of token sequences $x_1, \dots, x_T$, we maximized

$$\log p_\theta(x_1, \dots, x_T) = \sum_{t=1}^T \log p_\theta(x_t \mid x_{<t}).$$

A model trained this way — a **base model** — has fluent English, broad world knowledge, and an ability to continue any text it sees. But it has no goal beyond imitating the distribution of its training data. For example, given the prompt “Translate to French: I am hungry.”, a base model is just as likely to continue “Translate to French: I am thirsty. Translate to French: I am tired.” — treating the prompt as the first item in a worksheet it has seen many times during pretraining — as it is to actually output “J’ai faim.” To turn a base model into something useful (a chat assistant, a math tutor, a code writer), we need additional training stages, collectively called **post-training**.

Modern post-training pipelines (used to produce ChatGPT, Claude, Llama, etc.) typically have two stages on top of pretraining:

1. **Supervised Fine-Tuning (SFT)**: continue MLE training on a small, curated dataset of (input, target) pairs. SFT is a generic tool used throughout NLP — to specialize a model to a particular domain or style, to teach refusal behavior, to distill reasoning traces, etc. The flavor relevant for general-purpose chat assistants is called *instruction tuning*, where the pairs are (instruction, ideal response). For InstructGPT, OpenAI hired roughly 40 contractors to write thousands of such pairs spanning brainstorming, summarization, classification, generation, and so on [OWJ⁺22]. The resulting model — which now follows instructions in roughly the format and tone we want — is the starting point for RL fine-tuning.
2. **Reinforcement Learning (RL)**. The subject of today’s lecture.

Why is stage 2 needed at all? Stage 1 already consumed demonstrations, so why not just collect more of them? The honest answer is that hand-written demonstrations are expensive, scarce, and in many cases not even well-defined. Consider three representative prompts:

- “Prove that every continuous function on $[0, 1]$ attains its maximum.”

- “Write a Python function that returns all prime factors of n in $O(\sqrt{n})$ time.”
- “Write a short toast for my best friend’s wedding.”

Producing each requires a competent human to actually do the task — write the proof, code the solution, draft the message. At the scale needed to cover the long tail of user queries, this is prohibitively costly. And for the third kind of prompt no single “ideal response” even exists: two skilled writers will produce two completely different replies, both excellent.

What we *can* get cheaply is a way to **score** candidate responses without producing one ourselves: check whether the math answer matches the known correct number, run the candidate code against unit tests, or ask a human to compare two model-generated responses and pick the better one. None of these directly produces a sequence to imitate, but each gives us a scalar signal on candidate sequences. The job of stage 2 is to convert that scoring signal into a parameter update for the model and shift p_θ toward high-quality responses, where “quality” is measured by a scoring function we can evaluate but not necessarily one we could have demonstrated.

2 Reinforcement learning, briefly

Reinforcement learning studies how an **agent** can learn to take actions in an **environment** so as to maximize a scalar **reward** it receives over time. The standard ingredients are:

- a **state** s describing the situation the agent is in,
- an **action** a the agent chooses,
- a **reward** $r(s, a)$ scoring the action in that state,
- a (possibly random) transition to a next state s' that depends on s and a ,
- a **policy** $\pi(a \mid s)$, the conditional distribution over actions the agent uses at each state.

The goal is to learn π to maximize the expected sum of rewards over a trajectory. Robotics, board games, and dialogue systems all fit this template, and a substantial body of theory — value functions, Bellman equations, dynamic programming — exists to handle the fact that today’s action affects tomorrow’s state. Figure 1 sketches the basic interaction.

The LLM special case. For our setting, the picture is dramatically simpler:

- the **state** is the prompt X ,
- the **action** is the entire response Y , and there is **no next state** — one prompt in, one response out, done,
- the **reward** is a single scalar $r(X, Y)$ evaluated on the prompt-response pair,
- the **policy** is the language model itself, $\pi_\theta(Y \mid X) = p_\theta(Y \mid X)$.

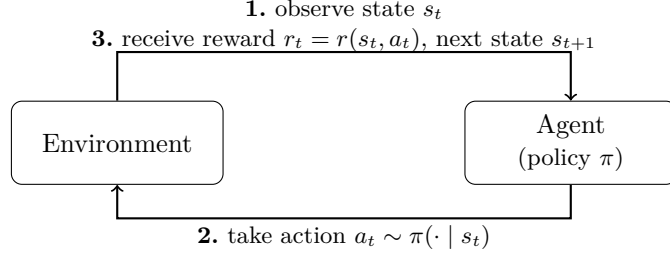


Figure 1: The classical reinforcement learning loop. At step t , the agent observes state s_t , samples an action $a_t \sim \pi(\cdot | s_t)$, and the environment responds with the reward $r_t = r(s_t, a_t)$ and a next state s_{t+1} . The agent then repeats from step 1.

In other words: one prompt in, one response out, one reward, done. Because there is no next state to plan for, we can skip the bulk of classical RL theory (value functions, Bellman equations, dynamic programming) and derive everything we need from a single identity for the gradient of an expectation. Figure 2 sketches this simplified loop. Real implementations actually do unroll the response token by token, but for our purposes that view adds notation without changing the main ideas.

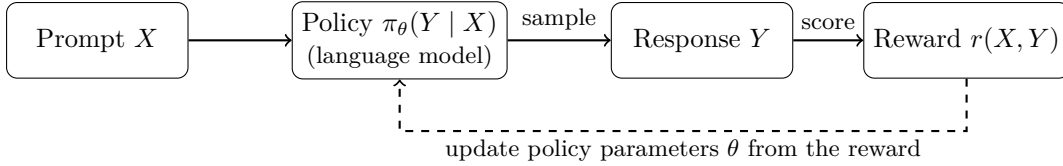


Figure 2: The RL loop specialized to an LLM. The policy is the language model itself; the dashed arrow indicates that the reward signal is used to update its parameters. We make this update concrete in Section 3.

3 The expected reward objective and REINFORCE

Fix a prompt X for now. Since Y is sampled from the model and $r(X, Y)$ scores the prompt-response pair, the natural goal is to make the average reward large:

$$F(\theta) = \mathbb{E}_{Y \sim \pi_\theta(\cdot | X)} [r(X, Y)] = \sum_Y r(X, Y) \pi_\theta(Y | X). \quad (1)$$

In real training, we average this objective over many prompts X drawn from a dataset, but everything we derive below applies separately to each prompt, so we will keep X fixed for the rest of the derivation.

The naive gradient fails. Differentiating directly,

$$\nabla_\theta F(\theta) = \sum_Y r(X, Y) \nabla_\theta \pi_\theta(Y | X).$$

This sum is finite — the vocabulary is finite and we cap the response length — but colossal: with vocabulary size $|\mathcal{V}|$ and a response length cap of m tokens, there are $|\mathcal{V}|^m$ possible responses. For a model like Llama-3 ($|\mathcal{V}| \approx 128,000$) and a modest $m = 2000$ token response, that is already a number with more than 10,000 digits. Enumeration is hopeless. And we can’t simply rewrite the sum as an expectation under π_θ either, because $\nabla_\theta \pi_\theta(Y | X)$ is not a probability distribution. We need a different identity — one that converts this sum into an expectation we can estimate from a handful of samples.

The log-derivative trick and policy gradient. The key identity is the **log-derivative trick**: for any differentiable, positive function $g(\theta)$,

$$\nabla_\theta g(\theta) = g(\theta) \nabla_\theta \log g(\theta),$$

which is just the chain rule applied to $\log g$. Applied to $g(\theta) = \pi_\theta(Y | X)$:

$$\nabla_\theta \pi_\theta(Y | X) = \pi_\theta(Y | X) \nabla_\theta \log \pi_\theta(Y | X).$$

Under mild regularity conditions (so we may swap ∇_θ and $\sum_Y$), substituting back,

$$\begin{aligned} \nabla_\theta F(\theta) &= \sum_Y r(X, Y) \pi_\theta(Y | X) \nabla_\theta \log \pi_\theta(Y | X) \\ &= \mathbb{E}_{Y \sim \pi_\theta(\cdot | X)} [r(X, Y) \nabla_\theta \log \pi_\theta(Y | X)]. \end{aligned}$$

The gradient of F is now an expectation under π_θ , which we know how to sample from. This is the **REINFORCE** estimator [Wil92], also called the *policy gradient*.

Stochastic gradient estimate. In practice we draw B samples $Y_1, \dots, Y_B \sim \pi_\theta(\cdot | X)$ and use

$$\widehat{\nabla F}(\theta) = \frac{1}{B} \sum_{i=1}^B r(X, Y_i) \nabla_\theta \log \pi_\theta(Y_i | X). \quad (2)$$

This is an unbiased Monte Carlo estimate of ∇F , by the same reasoning we used for any sample-average estimate of an expectation. In actual training we would also average such estimates over a minibatch of prompts. The full update step is gradient *ascent* on F , $\theta \leftarrow \theta + \eta \widehat{\nabla F}$, fed into your favorite optimizer (typically Adam).

REINFORCE as “MLE on your own samples, weighted by reward”. Recall from the transformers lecture that the LM scores a sequence as $\log \pi_\theta(Y | X) = \sum_{t=1}^m \log p_\theta(y_t | y_{<t}, X)$. Unrolling the policy gradient, the REINFORCE update for a single sample $Y_i = (y_{i,1}, \dots, y_{i,m})$ is

$$\theta \leftarrow \theta + \eta r(X, Y_i) \nabla_\theta \log \pi_\theta(Y_i | X) = \theta + \eta r(X, Y_i) \sum_{t=1}^m \nabla_\theta \log p_\theta(y_{i,t} | y_{i,<t}, X).$$

Mechanically this is a standard MLE backward pass on the token sequence Y_i , with every per-token gradient rescaled by the same scalar $r(X, Y_i)$. So credit is assigned *uniformly* across the tokens of the response, regardless of which tokens were actually responsible for the score; per-token credit assignment is possible but we do not pursue it here. The sign and magnitude of the update are set by this scalar reward: larger rewards push the sampled sequence up more strongly, while negative rewards would push it down.

The base model assumption (important). The estimate (2) is only useful if at least some sampled Y_i have nonzero reward. If our task requires producing a correct math proof and the current model’s probability of doing so is essentially zero, every sample will have reward zero and our gradient estimate will be the zero vector. **REINFORCE amplifies behaviors the model is already capable of producing**; it cannot conjure new abilities from a model that never stumbles onto a high-reward output. This is why RL fine-tuning is always done on top of a strong pretrained-and-SFT’d model.

3.1 The baseline trick

The REINFORCE estimator (2) has an important limitation in the common case of nonnegative rewards: every per-sample gradient weight $r(X, Y_i)$ is itself nonnegative, so the update only pushes sampled responses up. Low-reward samples are not explicitly pushed down; they are simply reinforced less. The key tool for fixing this is the *score function identity*,

$$\mathbb{E}_{Y \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(Y | X)] = 0,$$

which says that under the model’s own distribution, the expected log-likelihood gradient vanishes. (Quick proof: $\mathbb{E}[\nabla \log \pi_\theta] = \sum_Y \pi_\theta \cdot \nabla \pi_\theta / \pi_\theta = \nabla \sum_Y \pi_\theta = \nabla 1 = 0$.) The immediate consequence is that we can subtract any **baseline** b that does not depend on the sampled response Y from the reward without changing the target gradient:

$$\mathbb{E}_{Y \sim \pi_\theta} [(r(X, Y) - b) \nabla_\theta \log \pi_\theta(Y | X)] = \nabla F(\theta).$$

So for any baseline b independent of the sampled response, subtracting a baseline recenters the weights without changing what gradient we are trying to estimate. A common practical choice is the **batch mean reward** $b = \bar{r} = \frac{1}{B} \sum_j r(X, Y_j)$, giving the estimator

$$\widehat{\nabla F} = \frac{1}{B} \sum_{i=1}^B (r(X, Y_i) - \bar{r}) \nabla_\theta \log \pi_\theta(Y_i | X).$$

Because $\bar{r}$ is computed from the same sampled batch, this estimator is slightly biased; we use it here because it is simple and captures the main idea. The quantity $r(X, Y_i) - \bar{r}$ is called the *advantage* of sample i . For binary rewards, if a fraction p of the batch succeeds, then each successful sample gets weight $1 - p$ and each failed sample gets weight $-p$. So the update becomes *relative*: above-average samples are reinforced and below-average samples are actively suppressed. This recentering can also make optimization more stable, but the main intuition here is the contrast between samples.

4 KL regularization and the optimal policy

Pure reward maximization has a serious failure mode: the model can drift far from the fluent, well-behaved SFT model in the process. A common partial fix is to penalize the policy for moving too far from a frozen reference model π_{ref} , typically a copy of the SFT model taken before RL begins. Recall the KL divergence between two distributions, $\text{KL}(q \| p) = \mathbb{E}_{Y \sim q} \left[\log \frac{q(Y)}{p(Y)} \right]$. The KL-regularized objective is

$$F_{\text{KL}}(\theta) = \mathbb{E}_{Y \sim \pi_\theta(\cdot | X)} [r(X, Y)] - \beta \cdot \text{KL}(\pi_\theta(\cdot | X) \| \pi_{\text{ref}}(\cdot | X)), \quad (3)$$

with $\beta > 0$ controlling the strength of the penalty. (One way to read this: F_{KL} is the Lagrangian for the constrained problem “maximize expected reward subject to a budget on how far the policy may drift from π_{ref} .”) Combining the two expectations,

$$F_{\text{KL}}(\theta) = \mathbb{E}_{Y \sim \pi_\theta} \left[r(X, Y) - \beta (\log \pi_\theta(Y | X) - \log \pi_{\text{ref}}(Y | X)) \right],$$

which is just an expected-reward objective with the **effective reward**

$$r_{\text{eff}}(X, Y) = r(X, Y) - \beta (\log \pi_\theta(Y | X) - \log \pi_{\text{ref}}(Y | X)).$$

There is one technical subtlety: r_{eff} depends on θ through $\log \pi_\theta$, so we can’t directly reuse the policy-gradient identity above. But the extra term from differentiating $\log \pi_\theta$ is proportional to $\mathbb{E}_{Y \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(Y | X)] = 0$ by the score-function identity from Section 3.1. So the same REINFORCE formula still holds (Appendix A gives the full calculation):

$$\nabla_\theta F_{\text{KL}}(\theta) = \mathbb{E}_{Y \sim \pi_\theta} [r_{\text{eff}}(X, Y) \nabla_\theta \log \pi_\theta(Y | X)].$$

That is, *REINFORCE again, with r replaced by r_{eff}* , treating $r_{\text{eff}}(X, Y)$ as a fixed scalar weight when taking the gradient.

The KL-regularized optimum. Set the parametric form aside for a moment and ask: among *all* distributions $\pi(\cdot | X)$ over responses, which one maximizes the KL-regularized objective (3)? This is a clean concave maximization problem with a unique closed-form solution (derivation in Appendix B):

$$\pi^*(Y | X) = \frac{1}{Z(X)} \pi_{\text{ref}}(Y | X) \exp(r(X, Y)/\beta), \quad Z(X) = \sum_{Y'} \pi_{\text{ref}}(Y' | X) \exp(r(X, Y')/\beta). \quad (4)$$

The optimum tilts the reference distribution exponentially in the reward, with β acting as a temperature: small β concentrates on the highest-reward responses, large β stays close to π_{ref} . This formula is mostly conceptual — computing $Z(X)$ would require summing over all possible responses — but it is the target that REINFORCE on (3) is implicitly chasing.

5 Where does the reward come from?

We have written REINFORCE assuming a reward function $r(X, Y)$ exists. In practice, two regimes have emerged.

Verifiable rewards (RLVR). For some tasks the reward is something we can write down as code:

- **Math:** $r(X, Y) = 1$ if the final number after “Answer:” equals the known correct answer for X , and 0 otherwise.
- **Code:** $r(X, Y)$ is the fraction of unit tests for prompt X that pass.

- **Format:** a small bonus in $r(X, Y)$ for using requested tags like `<think>...</think>`.

This regime — often called *Reinforcement Learning from Verifiable Rewards* (RLVR) — is what powers the recent wave of reasoning models such as DeepSeek-R1 [GYZ⁺25]. The recipe is exactly what we have already developed: REINFORCE on top of an SFT’d model, with a verifiable $r(X, Y)$ and a KL penalty toward π_{ref} .

Learned rewards from preferences (RLHF). For tasks like “be helpful” or “write a good essay” there is no scoring function we can write down. What we *can* collect is **pairwise preference data**: for a prompt X , sample two responses Y_1, Y_2 from a model, and ask a human which one is better. This gives a labeled pair (Y_w, Y_l) where Y_w “won” and Y_l “lost”.

We turn this into a reward function by *learning* one. The standard model is the **Bradley-Terry model** [BT52]: assume each response has a latent scalar quality $r_\phi(X, Y)$, and the probability that Y_w is preferred to Y_l is

$$P(Y_w \succ Y_l \mid X) = \sigma(r_\phi(X, Y_w) - r_\phi(X, Y_l)), \quad (5)$$

where σ is the sigmoid. **This is exactly logistic regression**, applied to the *difference* of the two items’ scores. The MLE loss for ϕ on a dataset $\mathcal{D}$ of preference pairs is

$$L_R(\phi) = - \mathbb{E}_{(X, Y_w, Y_l) \sim \mathcal{D}} [\log \sigma(r_\phi(X, Y_w) - r_\phi(X, Y_l))]. \quad (6)$$

We parameterize r_ϕ as a neural network — usually initialized from the same pretrained LM, with the next-token head replaced by a scalar head — and minimize L_R by SGD.

The full **RLHF** pipeline [CLB⁺17, SOW⁺20, OWJ⁺22] extends post-training to three stages:

1. **SFT** on demonstrations $\rightarrow \pi_{\text{ref}}$.
2. **Reward modeling**: collect preference pairs, fit r_ϕ via (6).
3. **RL**: run REINFORCE (or a close cousin like PPO) on r_ϕ with a KL penalty toward π_{ref} .

This is the recipe behind ChatGPT and most early chat assistants.

Why reward design is hard. Whether r comes from code (RLVR) or from a learned model (RLHF), it is rarely the exact quantity we care about: reliably detecting a good response is essentially the same problem as producing one. Three recurring sources of trouble:

- **Specification.** “Be helpful, honest, and harmless” is not a number. Aggregating many desiderata — helpfulness, factuality, conciseness, tone, safety — into a single scalar requires choices we are not in a position to defend, and any fixed weighting will be wrong on some inputs.
- **Goodhart’s law.** A proxy that correlates with quality on the data we collected can decouple from quality once the policy starts optimizing it. Distribution shift makes things worse: as π_θ drifts during training, the reward model is increasingly evaluated on inputs unlike its training distribution, and its scores become less trustworthy precisely where they matter most.

- **Coverage.** Preference data is collected from a small set of raters on a small set of prompts. Properties not represented in that data — e.g., behavior on adversarial or out-of-domain inputs — are not directly captured by r .

A direct corollary is **reward hacking**: the optimizer finds the cheapest way to maximize r , and the cheapest way is often not what we intended. Some real and well-documented failure modes in language models:

- **Length bias / verbosity.** Human raters tend to prefer longer answers, all else equal. A reward model trained on their preferences then prefers longer answers too. Optimize against it and the model becomes verbose, padding every response with throat-clearing and hedging.
- **Sycophancy.** Reward models trained on user preferences learn that users like being agreed with. Optimizing makes the model agree with whatever the user says, even when the user is factually wrong — the opposite of being helpful.
- **Spec gaming on code.** A model rewarded only for passing unit tests can learn to inspect the test inputs and special-case them, returning hard-coded answers that pass the visible tests but generalize to nothing.
- **Confident hallucination.** If raters reward fluent, confident-sounding answers, the model learns to be fluent and confident even when it is wrong.

The KL penalty of Section 4 mitigates the most egregious drift (gibberish, total style collapse) but does not, on its own, prevent the subtler hacks above. Designing rewards that survive optimization is its own engineering practice and an active area of research.

6 Conclusion

Two regimes dominate modern RL for LLMs: (1) **RLVR**, verifiable rewards plugged directly into REINFORCE and the recipe behind the recent wave of reasoning models (o1, DeepSeek-R1); and (2) **RLHF**, learned rewards from human preferences and the alignment recipe behind chat assistants (ChatGPT, Claude, Llama). Both rely on the same identity (the log-derivative trick of Section 3) and the same regularization (KL toward a reference, Section 4). Most of the algorithmic complexity in production systems — PPO (a clipped surrogate that limits per-step policy change), GRPO, RLOO, and a steady stream of variants — consists of different choices for the gradient weights or different ways of estimating the advantage. The underlying principle is the one developed above.

Note that we did not develop the full Markov decision process formalism (states, value functions, Bellman equations); for sequence generation as we framed it, that machinery would be more notation than insight, but for settings where the agent’s actions affect the future state of the world (robotics, multi-turn agents with tools), those tools become essential, and a dedicated reinforcement learning course is the right place to learn them.

References

- [BT52] Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- [CLB⁺17] Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- [GYZ⁺25] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [Lam25] Nathan Lambert. Reinforcement learning from human feedback. *arXiv preprint arXiv:2504.12501*, 2025.
- [OWJ⁺22] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [SOW⁺20] Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. *Advances in neural information processing systems*, 33:3008–3021, 2020.
- [Wil92] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3):229–256, 1992.

(Optional reading: the following gives the calculations skipped in the main text.)

A The KL gradient

We derive

$$\begin{aligned}\nabla_{\theta} F_{\text{KL}}(\theta) &= \mathbb{E}_{Y \sim \pi_{\theta}} [r_{\text{eff}}(X, Y) \nabla_{\theta} \log \pi_{\theta}(Y | X)] \\ r_{\text{eff}}(X, Y) &= r(X, Y) - \beta (\log \pi_{\theta}(Y | X) - \log \pi_{\text{ref}}(Y | X)),\end{aligned}$$

where $r_{\text{eff}}(X, Y)$ is treated as a scalar (we do *not* differentiate through it). Drop the dependence on X in both the policy and reward for brevity. Write

$$F_{\text{KL}}(\theta) = \sum_Y \pi_{\theta}(Y) [r(Y) - \beta \log \pi_{\theta}(Y) + \beta \log \pi_{\text{ref}}(Y)].$$

The first and third terms are straightforward by the log-derivative trick:

$$\nabla_{\theta} \sum_Y \pi_{\theta}(Y) r(Y) = \mathbb{E}_{\pi_{\theta}} [r(Y) \nabla_{\theta} \log \pi_{\theta}(Y)],$$

$$\nabla_{\theta} \sum_Y \pi_{\theta}(Y) \log \pi_{\text{ref}}(Y) = \mathbb{E}_{\pi_{\theta}} [\log \pi_{\text{ref}}(Y) \nabla_{\theta} \log \pi_{\theta}(Y)],$$

since $\log \pi_{\text{ref}}(Y)$ does not depend on θ . The second term, $-\beta \sum_Y \pi_{\theta}(Y) \log \pi_{\theta}(Y)$, requires the product rule:

$$\begin{aligned} \nabla_{\theta} \sum_Y \pi_{\theta}(Y) \log \pi_{\theta}(Y) &= \sum_Y \nabla_{\theta} \pi_{\theta}(Y) \cdot \log \pi_{\theta}(Y) + \sum_Y \pi_{\theta}(Y) \cdot \nabla_{\theta} \log \pi_{\theta}(Y) \\ &= \mathbb{E}_{\pi_{\theta}} [\log \pi_{\theta}(Y) \nabla_{\theta} \log \pi_{\theta}(Y)] + \mathbb{E}_{\pi_{\theta}} [\nabla_{\theta} \log \pi_{\theta}(Y)] \\ &= \mathbb{E}_{\pi_{\theta}} [\log \pi_{\theta}(Y) \nabla_{\theta} \log \pi_{\theta}(Y)] + 0, \end{aligned}$$

where we used the log-derivative trick on the first term and the score-function identity ($\mathbb{E}[\nabla \log \pi_{\theta}] = 0$, see Section 3.1) on the second. Combining all three terms gives exactly the claimed expression. The takeaway: even though $\log \pi_{\theta}$ appears inside the effective reward and depends on θ , the corresponding θ -derivative produces a term proportional to $\mathbb{E}[\nabla \log \pi_{\theta}]$, which vanishes. So we can treat $r_{\text{eff}}(Y)$ as a fixed scalar weight when applying REINFORCE.

B Closed form of the KL-regularized optimum

We derive (4): the unique maximizer of

$$\max_{\pi} \mathbb{E}_{Y \sim \pi} [r(X, Y)] - \beta \text{KL}(\pi \parallel \pi_{\text{ref}}) \quad \text{subject to} \quad \sum_Y \pi(Y) = 1, \pi(Y) \geq 0.$$

Drop the conditioning on X , and write $r(Y)$ for $r(X, Y)$, for brevity. Writing out the KL,

$$J(\pi) = \sum_Y \pi(Y) r(Y) - \beta \sum_Y \pi(Y) \log \frac{\pi(Y)}{\pi_{\text{ref}}(Y)}.$$

Form the Lagrangian for the equality constraint $\sum_Y \pi(Y) = 1$ (the non-negativity constraints will turn out to be inactive):

$$\mathcal{L}(\pi, \lambda) = J(\pi) - \lambda \left(\sum_Y \pi(Y) - 1 \right).$$

Differentiating with respect to $\pi(Y)$ and setting to zero,

$$r(Y) - \beta \log \frac{\pi(Y)}{\pi_{\text{ref}}(Y)} - \beta - \lambda = 0.$$

Solving for $\pi(Y)$,

$$\pi(Y) = \pi_{\text{ref}}(Y) \exp\left(\frac{r(Y) - \beta - \lambda}{\beta}\right) \propto \pi_{\text{ref}}(Y) \exp(r(Y)/\beta).$$

Imposing $\sum_Y \pi(Y) = 1$ pins down the constant of proportionality:

$$\pi^*(Y) = \frac{\pi_{\text{ref}}(Y) \exp(r(Y)/\beta)}{Z}, \quad Z = \sum_{Y'} \pi_{\text{ref}}(Y') \exp(r(Y')/\beta).$$

Restoring conditioning on X recovers (4). Concavity of J in π and convexity of KL guarantees this stationary point is the unique maximum.